

Programmer's Guide

From topology to a verifiable root-cause engine: how this repository is put together, how the RootCause RAG pipeline was built and why, how to check its claims yourself, and how to stand the whole stack up on your own machine.

What this is not. NEXUS-1 is a Phase-0 *educational demonstrator* backing a book series. It is not safety-class or operational software and is not connected to any real facility. Every plant, component and incident in it is a teaching fixture.

Repository state described	<code>v1.0.0</code> branch at commit <code>1fbada7</code> (ADR-044), 2026-09-25
Grounded in	the repo's own ADRs (<code>docs/adr/</code>), runbooks (<code>docs/runbooks/</code>), evidence reports (<code>artifacts/evidence/</code>) and source code — plus live runs made while writing this guide
Source of this PDF	<code>docs/guide/nexus1-programmers-guide.html</code> (rendered by <code>docs/guide/build-guide-pdf.mjs</code>)
Companion script	<code>docs/guide/verify-audit-chain.ps1</code> — independent audit-chain verifier (Section 3)

Contents

How to read this guide

1 · Project topology

- 1.1 The shape in one picture
- 1.2 Seventeen bounded contexts, two runtime hosts, one gateway
- 1.3 Solution layout and the dependency law
- 1.4 The messaging backbone
- 1.5 The BFF and the Angular console
- 1.6 Ports, databases and logins at a glance
- 1.7 Topology gaps, named

2 · How the RAG was built, and why

- 2.1 The purpose, plainly
- 2.2 Where it started: *From Flood to Cause*
- 2.3 The fixed-incident-first decision
- 2.4 The One-Truth Pipeline
- 2.5 The ten anti-hallucination controls (H1–H10)
- 2.6 Semantic Kernel, on a short leash
- 2.7 Two worlds that must not be confused
- 2.8 The build, commit by commit
- 2.9 Where the engine stands today

3 · Verify the claims yourself

4 · Troubleshooting known setup friction

5 · The honest-gap discipline, as philosophy

6 · Installation and running guide

Appendices: [ADR index](#) · [RAG evidence index](#) · [glossary](#)

How to read this guide

This guide is written by one programmer for another. It assumes you are comfortable with C#/.NET, SQL Server and a terminal, and that you want to know *how the thing actually works* — not a marketing tour.

Three conventions run throughout:

- **VERIFIED** marks a statement that was checked against the live system or the repository while this guide was written (the evidence is named next to it).
- **GAP** marks something that is deliberately **not** built, only partially built, or known to be wrong in the repo's own docs. The project's standing rule is that a gap gets named, never hidden (Section 5 explains why).
- File paths are repo-relative. ADR numbers refer to `docs/adr/ADR-NNN-*.md`; evidence reports live in `artifacts/evidence/`.

Three corrections to the brief this guide was written from, made because the repository disagreed and the repository wins:

- There are **17 bounded contexts** (one per Schema Atlas sector). `Nexus1.ModularRuntime` composes **16** of them; the 17th, `RootCause`, runs in `Nexus1.RootCause.Host`. The BFF surfaces all 17 (16 in-process, `RootCause` over HTTP).
- The `RootCause/RAG` phase is **17 commits** since ADR-032, not sixteen: 15 feature slices plus 2 small corrective commits. All 17 are listed in §2.8.
- The provisioning runbook's migration command (`--startup-project src/Hosts/Nexus1.RootCause.Host`) **did not work** — the hosts do not reference `Microsoft.EntityFrameworkCore.Design`. The working form (§6.3) was verified on this machine, and the runbook was corrected in the same commit as this guide, together with two other runbook errors found while writing it (step order, stale seed counts).

1 Project topology

NEXUS-1 is a **hybrid**: a modular monolith that keeps most of the domain in one process, with exactly one bounded context — RootCause — extracted to its own independently deployed host. The book series argues for earning each extraction; this repository extracted only the one the architecture decision records justify.

1.1 The shape in one picture

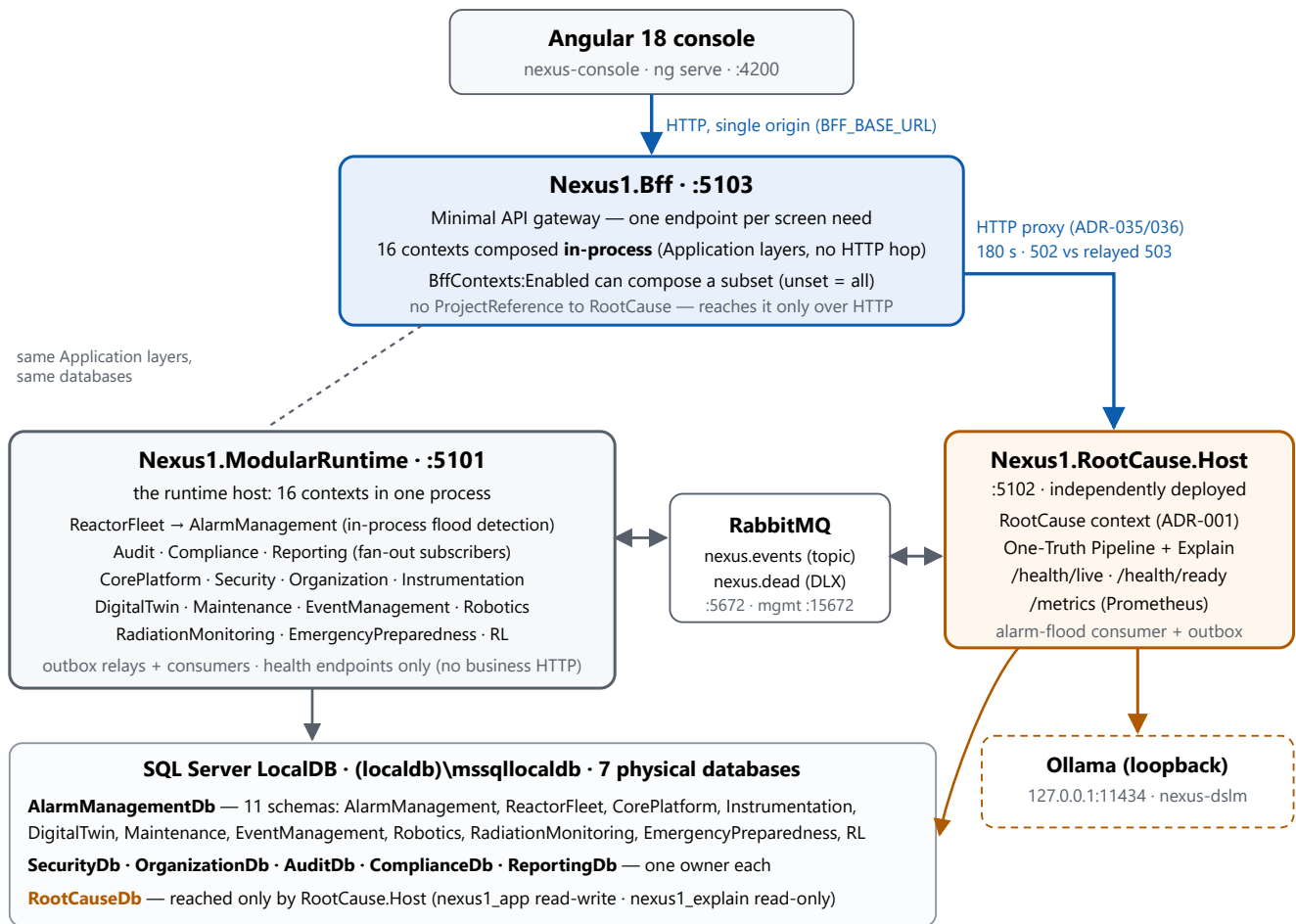


Figure 1 — Runtime topology as built. Solid arrows are real network or process hops; the dashed line marks that the Bff and ModularRuntime compose the same Application layers over the same databases.

Read the picture as three facts:

1. **Most of the domain is one deployable.** Sixteen contexts share the `Nexus1.ModularRuntime` process. ReactorFleet produces simulated telemetry that AlarmManagement consumes *in-process* to detect alarm floods (ADR-001-amend).
2. **RootCause is the one extraction.** `Nexus1.RootCause.Host` owns RootCauseDb outright, consumes `alarm-management.alarm-flood-detected.v1` from the broker, publishes its own events through a transactional outbox, and hosts the diagnosis (RAG) pipeline.
3. **The console talks to exactly one origin.** `Nexus1.Bff` composes the in-process contexts' Application layers directly and reaches RootCause only through an HTTP proxy. The browser never sees RootCause.Host.

1.2 Seventeen bounded contexts, two runtime hosts, one gateway

Each Schema Atlas sector is a bounded context with its own Domain / Application / Infrastructure projects under `src/Contexts/<Name>/`. The table shows where each one runs and where its data lives (from the host composition in `Program.cs` and each context's design-time `*DbContextFactory`).

Context	Runs in	Physical database	Phase / ADR
ReactorFleet	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 1 · ADR-001-amend, 003, 006
AlarmManagement	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 1 · ADR-004
RootCause	RootCause.Host only	RootCauseDb	Phase 1 · ADR-005; RAG ADR-032–044
Audit	ModularRuntime (+ BFF)	AuditDb	Phase 1 fan-out · ADR-010
Compliance	ModularRuntime (+ BFF)	ComplianceDb	Phase 1 fan-out · ADR-011
Reporting	ModularRuntime (+ BFF)	ReportingDb	Phase 1 fan-out · ADR-012
CorePlatform	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 1 · ADR-015
Security	ModularRuntime (+ BFF)	SecurityDb	Phase 2 sector 2 · ADR-016
Organization	ModularRuntime (+ BFF)	OrganizationDb	Phase 2 sector 3 · ADR-017
Instrumentation	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 4 · ADR-019
DigitalTwin	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 5 · ADR-020
Maintenance	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 6 · ADR-021
EventManagerment	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 7 · ADR-022
Robotics	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 8 · ADR-023
RadiationMonitoring	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 9 · ADR-024
EmergencyPreparedness	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 10 · ADR-025
ReinforcementLearning	ModularRuntime (+ BFF)	AlarmManagementDb	Phase 2 sector 11 · ADR-026

Two different hosts compose the same sixteen Application layers, for different jobs:

- **Nexus1.ModularRuntime** is the *runtime*: background services (outbox relays, broker consumers, retry dispatchers), plus `/health/live` and `/health/ready`. It exposes no business HTTP routes (ADR-007).
- **Nexus1.Bff** is the *read/command surface* for the console — Minimal API endpoints, one per screen need, dispatching to the same query and command handlers (hand-rolled dispatch, no MediatR, ADR-002-amend and ADR-030). It does not run the messaging background services: it registers no broker connection and composes AlarmManagement, Audit, Compliance and Reporting with their outbox relay and consumers switched off (`enableOutboxRelay: false`, `enableMessagingConsumer: false`).

The project's own evidence closes the Phase 2 vertical slices with: "*all seventeen Schema Atlas sectors, plus the Overview aggregation, are now live in* `Nexus1.Bff`" (`2026-08-23-bff-reinforcementlearning-policy-recommendations-slice.md`).

1.3 Solution layout and the dependency law

The layout is **context-first** (ADR-002, from *From Services to Runtime* Ch. 4): project-per-context-per-layer, never folders named "API/Worker/Database".

```

Nexus1.Runtime.sln · global.json (SDK 8.0.424, latestPatch) · Directory.Build.props · Directory.Packages.props
src/
  BuildingBlocks/ Nexus1.BuildingBlocks.{Domain,Application,Messaging,Observability}, Nexus1.ServiceDefaults
  Contexts/<Name>/ Nexus1.<Name>.{Domain,Application,Infrastructure} (+ Nexus1.Contracts.<Name> where it
publishes)
  Nexus1.RootCause.Explain # the ONLY project allowed a served-model package
Hosts/
  Nexus1.ModularRuntime · Nexus1.RootCause.Host · Nexus1.Bff
tests/
  ArchitectureTests · <Context>.UnitTests · <Context>.ComponentTests · ...
console/nexus-console/ Angular 18 + Jest + Playwright
docs/
  adr/ · runbooks/ · observability/ · guide/
artifacts/evidence/ one evidence report per slice (what was run, and its real output)

```

The dependency law is **enforced by** `Nexus1.ArchitectureTests`, which parses every `.csproj`'s `<ProjectReference>` graph (not compiled IL) and fails the test gate on violation:

- Inside a context, dependencies point inward: Infrastructure → Application → Domain.
- Across contexts, a project may reference only another context's `Nexus1.Contracts.*` project — never its Domain, Application or Infrastructure.
- Domain never references transport, persistence, hosts or contracts (domain events are not integration events).
- Hosts are composition roots only.
- **Served-model packages** (Semantic Kernel, the Ollama connector) may appear only in `Nexus1.RootCause.Explain` (ADR-032/033).
- **No passwords** in any `appsettings*.json` under `src/` (ADR-044).

1.4 The messaging backbone

Integration events travel through RabbitMQ with a **transactional outbox** on the producer side and an **inbox** (idempotent consumption) on the consumer side (ADR-008), plus retry with backoff and dead-lettering (ADR-009). All publishes go to the durable topic exchange `nexus.events`; dead letters go to `nexus.dead`. Queues are quorum queues; dead-lettering is applied as a broker *policy* through the management API at consumer start.

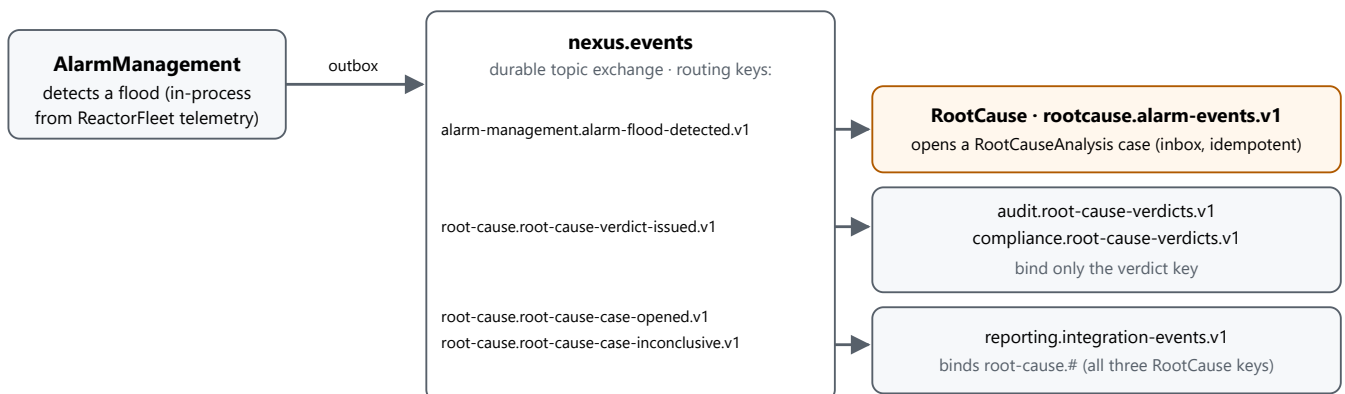


Figure 2 — The Phase 1 distributed slice: AlarmManagement → RootCause, with Audit, Compliance and Reporting as independent subscribers (ADR-001, 008, 010–012, 039).

Startup behaviour (ADR-044). Both hosts that talk to the broker now *start and retry* when it is unreachable: the connection is made by a background loop with backoff (1 s doubling to a 30 s cap), and each consumer's start sequence is retried through the shared `ConsumerStartup` helper. Before ADR-044, a missing broker crashed ModularRuntime (~9.7 s) and RootCause.Host (~16 s) with an unhandled `BrokerUnreachableException`. Readiness never depends on the broker — no HTTP route uses it.

1.5 The BFF and the Angular console

`console/nexus-console` is an Angular 18 application (Jest unit/component tests, Playwright end-to-end tests). Every API client targets a single constant, `BFF_BASE_URL = 'http://localhost:5103'`. The BFF enables CORS only in the Development environment and only for `http://localhost:4200`.

For RootCause, the BFF adds two thin proxies — `POST /api/v1/root-cause/incidents/{id}/diagnoses` (ADR-035) and `GET .../{id}/graph` (ADR-036) — that relay the host's status, body and content type *verbatim*. If RootCause.Host itself cannot be reached, the BFF returns **502**; if the host ran but a dependency of its pipeline failed, the host's own **503** is relayed. That distinction is deliberate and diagnostic.

The RootCause screens (Ch. 29) are `/rcgraph` (Root Cause Graph: verdict, ranked candidates, citations, audit hash, and the drawn fault tree) and `/incident` (Incident Analysis: the top cause). Both read one shared `RootCauseService`, so one diagnosis run feeds both — a property the live `@slow` Playwright test asserts end to end.

1.6 Ports, databases and logins at a glance

Process	Address	Health / notes
Angular console	<code>http://localhost:4200</code>	<code>npm start</code> (= <code>ng serve</code>)
Nexus1.Bff	<code>http://localhost:5103</code>	<code>/health/live</code> , <code>/health/ready</code>
Nexus1.RootCause.Host	<code>http://localhost:5102</code>	<code>/health/live</code> , <code>/health/ready</code> , <code>/metrics</code>
Nexus1.ModularRuntime	<code>http://localhost:5101</code>	<code>/health/live</code> , <code>/health/ready</code>
RabbitMQ	AMQP <code>:5672</code> · management <code>:15672</code>	management port is hard-coded to 15672 in <code>RabbitMQOptions</code>
Ollama	<code>http://127.0.0.1:11434</code>	loopback only; <code>/api/tags</code> lists models
Prometheus / Grafana (optional)	<code>:9090</code> / <code>:3000</code>	RootCause.Host metrics only (ADR-038)
OTel Collector (optional)	OTLP <code>:4317/4318</code> · health <code>:13133</code>	tracing/messaging-metrics evidence campaigns

The running hosts never use the developer's own sysadmin connection. They connect as `nexus1_app` (read/write DML on exactly the schemas they own, no DDL — ADR-028). RootCause's retrieval and validator path uses `nexus1_explain` (read-only, explicit `DENY` on every write verb — ADR-033/034). Migrations and provisioning use the developer's own Windows login through the design-time factories.

1.7 Topology gaps, named

- **No container images for the .NET hosts.** Only the console has a `Dockerfile`. A Kubernetes-readiness investigation (before ADR-044) found the health endpoints are now correct, but no image, manifests or non-LocalDB database target exist.
- **No authentication/authorization** on any HTTP surface beyond the development status quo.
- **The BFF has no observability** (no tracing/metrics); edge metrics such as `nexus1.edge.requests` are deferred (ADR-007/030/038).
- **Two test projects are empty shells:** `Nexus1.Contracts.ContractTests` and `Nexus1.DistributedSlice.EndToEndTests` contain no `.cs` files and report "No test is available".
- **The console's README is the Angular CLI boilerplate** (it mentions Karma; the project uses Jest and Playwright). The real E2E instructions are in `console/nexus-console/e2e/README.md`.

2 How the RAG was built, and why

2.1 The purpose, plainly

RootCause's diagnosis pipeline is a deterministic, grounded, auditable root-cause engine in which a language model only narrates a decision it never makes itself.

When an alarm flood hits, the engine decides which component started it by walking an engineered causal graph, checks that decision against the physical telemetry, and grounds it in a retrieved corpus. Only then is a small local model allowed to *explain* the already-made decision — in a strict output schema, citing only passages that were actually retrieved, naming only components that exist. Anything that fails a gate produces a *named abstention* instead of a verdict. Every outcome, verdict or abstention, is sealed into a SHA-256 hash chain that anyone can recompute (Section 3).

The design inverts the usual "ask the LLM what happened" RAG. Here the LLM is the least trusted component in the system, and the architecture is built so that it structurally cannot change the answer.

2.2 Where it started: *From Flood to Cause*

The RAG phase began with a close reading of *From Flood to Cause*, a NEXUS-1 companion book that describes the "One-Truth Pipeline": a deterministic engine decides, a served language model explains, and ten anti-hallucination controls (H1–H10) make confident fabrication structurally hard. Its Chapters 8–9 give the C#/SQL production form, Appendix C the grounding schema, and Figure 2.1 the worked incident **EVT-2026-0418** as a causal fault-propagation graph with delay windows. Chapter 10 works three case studies: 0418 (a cascade), 0419 (a measurement artefact) and 0420 (a QA escape).

Two constraints shaped everything that followed (ADR-032):

- **The book is not in the repository.** It is outside the project's seven-source authority list, and ADR-029 purged licensed source PDFs from the repo and its history. Every value the book supplies is therefore *pinned* — in the ADRs and in `GroundingSeed` — so the implementation traces to a recorded decision rather than to a PDF the repo does not carry. Figure 2.1's delay windows were re-derived from the book (`pdftotext -table`) and cross-checked arithmetically.
- **The book targets SQL Server 2025 preview features** (`VECTOR(1024)` , `VECTOR_DISTANCE` , `AI_GENERATE_EMBEDDINGS`). The available database is SQL Server 2016 (LocalDB 13.0 SP1). So cosine similarity is computed in C#, and embeddings are stored as JSON in `Corpus.EmbeddingJson`.

2.3 The fixed-incident-first decision

Instead of building a general question-answering assistant, the project built **one fixed incident end to end** — EVT-2026-0418 — as a walking skeleton, and only then added depth. The reasoning, recorded across ADR-032 to ADR-034:

- **A single known incident has a known right answer.** The book's own figure says FV-104 explains 11 of 14 alarms. If the engine computes 11/14 from seeded alarm counts, that is evidence; if it merely prints it, it is theatre.
- **It exposes every seam at once.** Graph walk, telemetry corroboration, retrieval, generation, validation and sealing all have to exist for one incident to go through. Nothing can hide behind "later".
- **It refuses premature generality.** A second incident (0419) was added only after the first was proven over HTTP, in the browser and in a live E2E test. The third (0420) is deliberately *not* solvable by the engine, and the repo says so.

- It gives the generation stage nowhere to hide. ADR-032 shipped the deterministic half with the model stage *visibly unbuilt* — no stub, no simulated narrative — until the hardware for a local model existed (ADR-033).

2.4 The One-Truth Pipeline

`FixedIncidentDiagnosisRunner` (Application layer, LLM-free) composes six stages behind interfaces. Each stage either passes the run forward or ends it with a named abstention.

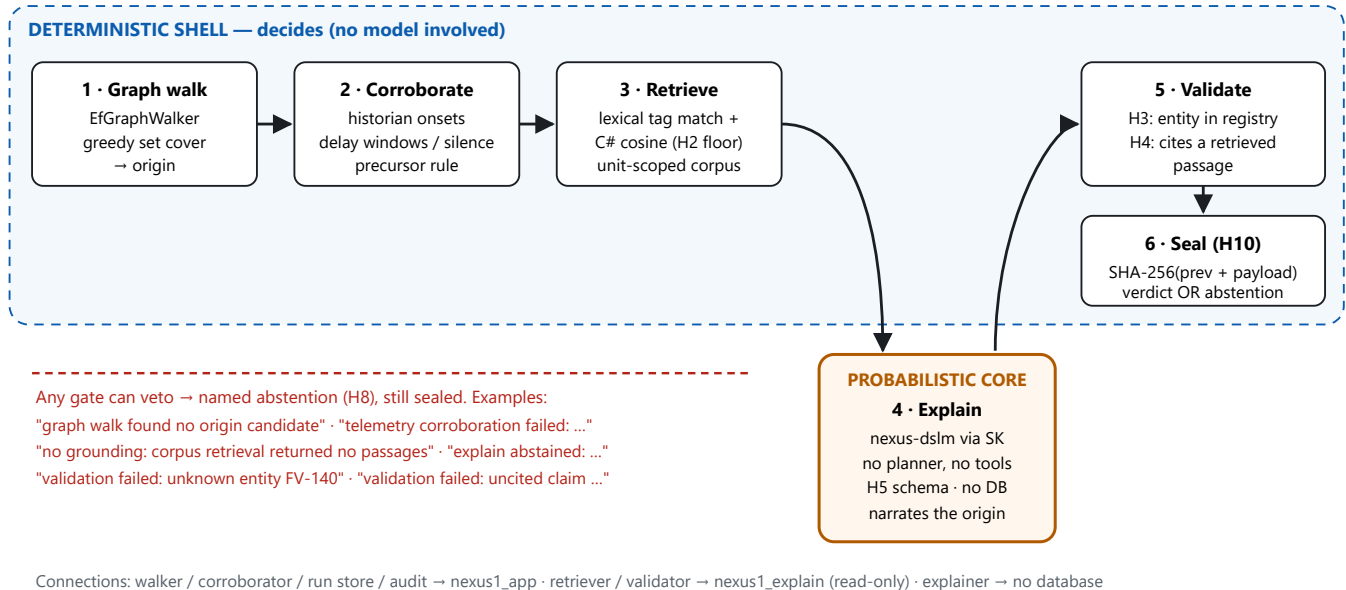


Figure 3 — The One-Truth Pipeline: deterministic shell around a probabilistic core. The engine decides the origin before the model runs; the model cannot change it.

What each stage actually does, in the code as it stands (ADR-032, 037, 042, 043):

- 1. Graph walk** (`EfGraphWalker`). Candidates are every component that is alarmed in the flood or is the source of an authored edge. Each candidate's explanation is the set of alarmed nodes reachable over *backbone* and *artefact* edges; learned edges are subordinate and rejected edges are never walked. Ranking is a greedy set cover (most not-yet-explained alarms first; ties by earlier historian onset, then lower `ComponentId`). Weight is an attribution share. Roles are structural: origin, downstream, parallel, contributing, independent, ruled-out. *Nothing is read from seeded "answer" fields* (ADR-042).
- 2. Corroborate** (`EfTelemetryCorroborator`). Cascade mode: every backbone edge on the origin's forward path must fit its delay window against historian onsets. Artefact mode: coupled channels deflect together, and independent witnesses must stay *flat* — silence is the evidence. Both modes then apply the **unexplained-precursor rule**: an alarmed component the origin does not reach, with a strictly earlier onset, means the graph is probably missing a coupling, so the run abstains (ADR-043).
- 3. Retrieve** (`EfRetriever`). Exact lexical tag match plus a C# cosine path over `nomic-embed-text` embeddings with an H2 relevance floor of 0.5. The corpus is scoped by unit, so incidents never ground on each other's passages.
- 4. Explain** (`SemanticKernelExplainer` in `Nexus1.RootCause.Explain`). One chat call to the pinned local model `nexus-dslm`, parsed into a strict JSON schema. A refusal or an unparseable reply is an abstention — never salvaged.
- 5. Validate** (`RegistryAntiHallucinationValidator`). H3: every named entity must exist in the component registry. H4: every claim must cite a passage that was actually retrieved.
- 6. Seal** (`Sha256AuditChainWriter`). The run's inputs, conclusion, candidates, citations and validated answer are serialized to JSON and appended to `RootCause.Audit`, where `Hash = SHA-256(PrevHash + Payload)` from a 64-zero genesis.

2.5 The ten anti-hallucination controls (H1–H10)

Control	What it guarantees	Where it lives / status
H1	Closed world: the model uses only the supplied context.	The <code>SYSTEM</code> block of <code>nexus-dslm.Modelfile</code> travels with the pinned model.
H2	Relevance floor: below it there is nothing to ground on.	<code>EfRetriever</code> , cosine floor 0.5 (a demonstrator value, not H9-calibrated).
H3	Every named entity exists in the registry.	<code>RegistryAntiHallucinationValidator</code> ; caught live by the H9 trap <code>trap-transposed-entity</code> (FV-140).
H4	Every claim cites a retrieved passage.	Same validator; H9 trap <code>trap-invented-source</code> (a real but unretrieved document).
H5	Output must fit a strict schema.	<code>StructuredAnswerParser</code> ; the schema lives in the Modelfile.
H6	Determinism.	The decision is deterministic (proven model-free). The narration is not byte-identical on CPU even at temperature 0 with a fixed seed — reported honestly (ADR-033).
H7	No write path for the model side.	The explainer has no database at all; retrieval runs as read-only <code>nexus1_explain</code> (proven live via <code>sys.dm_exec_sessions</code> , ADR-034).
H8	Abstain rather than guess; never salvage a bad draft.	Runner gates; abstention is a first-class <code>200</code> over HTTP.
H9	Golden and trap cases run on every change.	PARTIAL ADR-041 harness: goldens, two traps, a live tier. The leading-question trap is authored but skipped by name: no CauseTag-vs-origin check exists yet.
H10	Tamper-evident audit.	<code>Sha256AuditChainWriter</code> ; independently verifiable (Section 3).

2.6 Semantic Kernel, on a short leash

The model stage uses Microsoft Semantic Kernel 1.50.0 with the Ollama connector (1.50.0-alpha) — pinned below the latest release because 1.80.x pulls the .NET 10 `Microsoft.Extensions.*` wave into a .NET 8 solution (NU1605). The constraints are the point:

- `FunctionChoiceBehavior.None()` — no planner, no automatic tool calling. The model cannot call functions, query anything or decide which steps to run.
- **Temperature 0, seed 42, one call** (`GetChatMessageContentAsync`), with the determinism parameters pinned in the Modelfile rather than left to the caller.
- **The model is told the answer.** The prompt carries the origin the engine already chose; Modelfile rule 1 says `cause` must be exactly that tag, and rule 2 restricts `entities` to that one tag. The model's job is to explain, not to decide.
- **No database dependency.** `SemanticKernelExplainer` receives passages and returns a draft, so it structurally cannot read or write the store.
- **Air-gapped by configuration.** Ollama binds `127.0.0.1`. During ~70 s of continuous inference, zero non-loopback connections were observed from any Ollama process (ADR-033 evidence).
- **One project holds it all.** An architecture test fails the build if a served-model package appears anywhere except `Nexus1.RootCause.Explain`.

```
# src/Contexts/RootCause/Nexus1.RootCause.Explain/nexus-dslm.Modelfile (excerpt)
FROM qwen2.5:3b-instruct-q4_K_M
PARAMETER temperature 0
PARAMETER seed 42
PARAMETER top_p 1.0
PARAMETER repeat_penalty 1.0
PARAMETER num_ctx 8192
SYSTEM """"... Closed-world rule (H1): use ONLY the CONTEXT ... The deterministic engine has ALREADY
decided the origin; your only job is to explain that origin from the CONTEXT. ..."""
```

2.7 Two worlds that must not be confused

RootCause holds two separate records, and newcomers routinely conflate them:

	The engine's record: <code>DiagnosisRun</code>	The human case: <code>RootCauseAnalysis</code>
Created by	The diagnosis route (<code>POST .../diagnoses</code>), every call	An alarm flood event arriving from the broker, or provenance provisioning (0420)
Lifecycle	Immutable result: verdict or named abstention, plus candidates and an audit link	Open → Closed (with a verdict) or Open → Inconclusive (reason only, ADR-039)
Who decides	The deterministic engine	A human investigator
Published as events?	No	Yes: case-opened, verdict-issued, case-inconclusive (outbox → broker)
Incidents	EVT-2026-0418 and 0419 only (<code>IncidentRegistry</code>); anything else is <code>404</code>	Any flood; plus the fixed 0420 case (AnalysisId 20260420)

2.8 The build, commit by commit

The RootCause/RAG phase is 17 commits on top of the finished Phase 1 slice and the Phase 2 sectors. Each was its own planned slice with an ADR (or a stated reason for none), an evidence report, and a commit only after review.

#	Commit	Slice	What it proved
1	<code>4172674</code> 09-07	Deterministic engine · ADR-032	The whole LLM-free half on SQL Server 2016: grounding store, the four seams, audit chain, <code>GroundingSeed</code> . FV-104 computed as origin covering 11 of 14 alarms from the data; four fault injections each produce a named abstention. The generation stage was left visibly unbuilt — no stub.
2	<code>a8c3d1d</code> 09-22	Air-gapped Ollama stage · ADR-033	<code>Nexus1.RootCause.Explain</code> , the pinned <code>nexus-dslm</code> Modelfile, constrained SK, nomic embeddings, the read-only login. Loopback-only binding with zero egress captured. Honest H6 finding: decision deterministic, narration not.
3	<code>7d2856c</code> 09-22	Real HTTP route · ADR-034	<code>POST .../diagnoses</code> : an abstention is a <code>200</code> , infrastructure failure a <code>503</code> . The response's <code>auditHash</code> equals the newest <code>RootCause.Audit</code> row; the read-only split was proven live.
4	<code>40bfc7a</code> 09-22	BFF gateway hop · ADR-035	The BFF's first outbound call: a raw relay (no shared DTO), a 180 s timeout, and 502 (hop failed) kept distinct from a relayed 503 (host ran, dependency failed).
5	<code>78aee13</code> 09-22	<i>Fix</i> · ADR-033 addendum	A real cold call exceeded SK's default 100 s HttpClient timeout. The chat call now uses its own 150 s client — below the BFF's 180 s, so the host fails first with its own clean 503.
6	<code>66e1491</code> 09-23	Angular screens (Ch. 29)	Root Cause and Incident Analysis screens on the real pipeline through the BFF, with one shared service. Graph drawing was deferred with an explicit "pending a backend endpoint" note rather than faked.
7	<code>8afb19</code> 09-23	<i>Fix</i> · copy only	The Ch. 24 AI Diagnostics screen still claimed no grounded pipeline existed — now false. The wording was corrected: "nothing claims to exist that does not" cuts both ways.

#	Commit	Slice	What it proved
8	212b669 09-23	Graph topology endpoint · ADR-036	<code>GET .../graph</code> serves the seeded topology; the console draws Figure 29.1 with its existing inline-SVG renderer. Topology is 100% backend data; only screen positions are client constants.
9	47e3602 09-24	Live E2E (Ch. 33)	A <code>@slow</code> Playwright test drives console → BFF → RootCause.Host → Ollama → RootCauseDb and asserts both screens name the same origin, plus a spec-of-specs proving the assertion is load-bearing.
10	a5e0a6a 09-24	Second incident EVT-2026-0419 · ADR-037	<code>IncidentRegistry</code> , artefact-edge walking, silence-as-corroboration (flat witnesses confirm an EMI artefact), and unit-scoped corpora so incidents never cross-ground.
11	680d610 09-24	Observability · ADR-038	Appendix I's four diagnosis metrics on RootCause.Host's <code>/metrics</code> , plus local Prometheus and Grafana. <code>grounding_hit</code> is named a proxy, never "recall".
12	f86d4b5 09-24	Inconclusive third state · ADR-039	Investigators can terminally record "cannot conclude" (reason only, verdict stays null), published as <code>RootCauseCaseInconclusiveV1</code> and projected by Reporting — which binds <code>root-cause.#</code> and would otherwise have poisoned the event.
13	910a0bd 09-24	EVT-2026-0420 · ADR-040	The QA escape (CR-7 / WV-318) recorded as an Inconclusive, provenance-originated case with a nullable flood id. Deliberately <i>not</i> in the engine's registry: its 404 is the evidence of where the engine's reach ends.
14	cfc280a 09-24	H9 evaluation harness · ADR-041	JSON golden and trap cases with a deterministic tier and a skippable live tier, plus a discovery guard. The missing CauseTag-vs-origin check was found and its trap skipped by name.
15	7c59f9a 09-25	De-circularization · ADR-042	The walker stopped reading seeded answer-key fields. A poisoned-seed test proves byte-identical output; on the old walker, poisoning made PB-2A the origin. A pure-observable fixture is ranked correctly by hand-worked rules.
16	b4f90b7 09-25	Model-mismatch scenario · ADR-043	A graph missing an upstream coupling (IA-1 → AOV-2) used to seal a confident wrong verdict. The unexplained-precursor rule now abstains. Two controls prove the rule is not arbitrary; the classes it still cannot catch are listed.
17	1fbada7 09-25	Startup / readiness / secrets hardening · ADR-044	Hosts start and retry without a broker; readiness now covers the read-only connection and Ollama (reachable, not warm); connection strings moved to User Secrets; error details stay server-side under a trace id. It also found and fixed a migrations-history mismatch in the read-only context.

2.9 Where the engine stands today

Live on 2026-09-25 while writing this guide **VERIFIED**: a diagnosis of EVT-2026-0418 returned **FV-104** (origin, attribution weight 0.79, coverage 11/14) with RCP-1B parallel (0.14), BUS-2A contributing (0.07), six downstream symptoms and FT-7 ruled out, citing the book's Ch.1-2 and Ch.2 passages. EVT-2026-0419 returned **BKR-2A** (weight 1.00) with FT-9, LT-3 and PT-7 downstream and LOF-1 ruled out. EVT-2026-0420 returned `404 unknown incident`.

Named gaps in the RAG as of this commit

- **True `retrieval_recall` (Appendix J) is not measured.** Three incidents are far too small a labelled set; `grounding_hit` is a proxy (ADR-038, 041).
- **No CauseTag-vs-origin check.** A draft naming a registered, retrieved but wrong component (e.g. RCP-1B for 0418) would pass H3/H4; the H9 leading-question trap is skipped by name until that engine slice exists.
- **No free-form question interface.** The route takes an incident id; the "DSLM advisory" in the console's AI Diagnostics screen is labelled "not running in this build".
- **Audit and Compliance do not consume `RootCauseCaseInconclusiveV1`.** They bind only the verdict key, so they safely never receive it. The extension is the named follow-on of ADR-040.
- **Mismatch classes the precursor rule cannot catch** (ADR-043): a silent true cause, a spurious edge whose timing fits, a missing downstream edge; and a standing alarm with an early onset would cause a false abstention.
- **0418's delay windows are exact points** (min = max), so one second of real jitter would make it abstain. Recorded, not changed.
- **The weights are attribution shares, not probabilities.** The console still labels them "Illustrative weight" / "Illustrative confidence" — a wording mismatch left from before ADR-042, named for a UI slice.

3 Verify the claims yourself

This project's standing rule is *evidence over assertion*. This section applies that rule to the guide itself: instead of trusting that the audit chain works, you run one fixed incident and watch the hash in the HTTP response match the newest row in `RootCause.Audit` — then recompute it yourself, with no project code involved.

Everything below was run exactly as written on 2026-09-25 against this commit. The sample outputs are from that run. Your hashes will differ — they depend on your chain's history and on the model's narration — but the *relationships* must hold.

3.1 What you need running

Only three things: LocalDB with RootCauseDb migrated and provisioned, Ollama with `nexus-dslm` and `nomic-embed-text`, and `Nexus1.RootCause.Host`. Since ADR-044 the host starts without RabbitMQ, and the diagnosis route never uses the broker. If you have not installed anything yet, do Section 6 first, then come back.

```
# Start RootCause.Host. launchSettings.json supplies http://localhost:5102 and the
# Development environment (which is what loads User Secrets).
dotnet run --project src/Hosts/Nexus1.RootCause.Host

# In another shell: is it ready? (checks both SQL logins and Ollama)
curl http://localhost:5102/health/ready      # → Healthy
```

If readiness says `Unhealthy`, the host's console log names the failing check (`rootcause-db`, `rootcause-explain-db` or `ollama`). While writing this guide, the first attempt did exactly that: Ollama had stopped, readiness reported `ollama ... did not answer /api/tags`, and the diagnosis returned a generic `503` with a `traceId` instead of internals — the ADR-044 behaviour working as designed.

3.2 Run one fixed incident and note the hash

```
curl -X POST http://localhost:5102/api/v1/root-cause/incidents/EVT-2026-0418/diagnoses
```

Expect 20–150 s. A cold model load on a CPU is the slow part; see §4.3. Sample response from the author's run (trimmed):

```
HTTP 200 in 105.1 s
{ "diagnosisRunId": 41, "incidentId": "EVT-2026-0418", "verdict": "FV-104", "abstained": false,
  "candidates": [ { "tag": "FV-104", "role": "origin", "weight": 0.7857, "coverage": 0.7857 },
                  { "tag": "RCP-1B", "role": "parallel", "weight": 0.1429, ... }, { "tag": "BUS-
2A", "role": "contributing", ... },
                  { "tag": "SG-1", "role": "downstream", "weight": 0, ... }, ..., { "tag": "FT-7", "role": "ruled-out", ... } ],
  "citations": [ { "sourceLabel": "From Flood to Cause -- NEXUS-1 Companion, worked example, Ch.1-2",
                  { "sourceLabel": "From Flood to Cause -- NEXUS-1 Companion, worked example, Ch.2 (delay
windows)" } ],
  "auditHash": "1bd5bcfb3ba404c1cf068c2c60e0f0115da162b91fa7c170f69a2dc2df5c646" }
```

3.3 Look at the newest audit row with your own eyes

```
sqlcmd -S "(localdb)\mssqllocaldb" -E -d RootCauseDb -W -Q "SELECT TOP 1 Seq, Hash, PrevHash FROM
RootCause.Audit ORDER BY Seq DESC"
```

Seq	Hash	PrevHash
---	----	-----
41	1bd5bcfbb3ba404c1cf068c2c60e0f0115da162b91fa7c170f69a2dc2df5c646	c977055964e783eb598d9793ac2236d9685dc5a4a5eb6049b267ef3319cad5a1

The row's `Hash` is the `auditHash` you just received. Its `PrevHash` is the previous row's hash — that is the chain.

3.4 Recompute it yourself — the whole chain

Matching two strings proves the route returned what it stored. It does not prove the stored hash is honest. So recompute it: the formula is lowercase hex `SHA-256(UTF-8(PrevHash + Payload))`, starting from sixty-four zeros (`Sha256AuditChainWriter.ComputeHash`). The companion script `docs/guide/verify-audit-chain.ps1` does that for every row. It uses only Windows PowerShell's built-in `System.Data.SqlClient` and your own Windows login — no project assemblies, no SQL-login secrets.

```
.\docs\guide\verify-audit-chain.ps1 -ExpectedLatestHash
1bd5bcfbb3ba404c1cf068c2c60e0f0115da162b91fa7c170f69a2dc2df5c646
```

```
Rows checked: 41    broken links/hashes: 0
Newest row:   Seq 41 at 2026-09-25 09:03:35Z
  stored      Hash = 1bd5bcfbb3ba404c1cf068c2c60e0f0115da162b91fa7c170f69a2dc2df5c646
  recomputed Hash = 1bd5bcfbb3ba404c1cf068c2c60e0f0115da162b91fa7c170f69a2dc2df5c646
  payload (first 160 chars): {"IncidentId":"EVT-2026-0418","Verdict":"FV-104","AbstainReason":null,...
HTTP auditHash matches newest row: YES
CHAIN INTACT
```

The heart of the script is four lines — short enough to audit before you run it:

```
function Get-ChainHash([string]$prev, [string]$payload) {
    $bytes = [System.Text.Encoding]::UTF8.GetBytes($prev + $payload)
    ($sha.ComputeHash($bytes) | ForEach-Object { $_.ToString("x2") }) -join ""
} # each row: PrevHash == previous row's Hash, and Hash == Get-ChainHash(PrevHash, Payload)
```

Now try to break it. In a throwaway copy of the database, edit one character of any row's `Payload` and rerun the script: that row's recomputed hash no longer matches, and the tampering is reported at its exact `Seq`. That is what "tamper-evident" means here — not tamper-proof, but impossible to alter silently.

3.5 More claims you can check in minutes

Claim	How to check it
The decision is deterministic; the narration is not (H6)	POST 0418 twice. Both return FV-104 with identical candidates; the two <code>auditHash</code> values differ because the sealed payload includes the model's prose. Rerun the verifier: the chain still links.
The engine's reach ends at 0420	<code>curl -X POST .../incidents/EVT-2026-0420/diagnoses</code> → <code>404 {"error":"unknown incident 'EVT-2026-0420'"}</code> . The human case exists as Inconclusive: <code>SELECT Status, InconclusiveReason FROM RootCause.RootCauseAnalysis WHERE RootCauseAnalysisId = 20260420</code> .
The artefact incident is ruled by silence	POST 0419 → BKR-2A. The corroborator passes <i>because</i> the witnesses RTD-1 and NFX-1 have no historian deflection (ADR-037).

Claim	How to check it
The model side cannot write (H7)	Connect as <code>nexus1_explain</code> : <code>SELECT COUNT(*) FROM RootCause.Corpus</code> succeeds; any <code>INSERT</code> fails with Msg 229 (docs/runbooks/local-explain-readonly-sql-login.md).
Readiness tells the truth	Stop Ollama, then <code>curl .../health/ready</code> → <code>503 Unhealthy</code> ; the log names <code>ollama</code> . Unload the model with <code>ollama stop nexus-dslm</code> (server still up) → readiness stays <code>200</code> ; reachable is not warm (ADR-044).
Errors don't leak internals	With Ollama down, the diagnosis <code>503</code> body carries only a generic detail and a <code>traceId</code> ; search the host log for that id to find the full exception.
The tests are real	<code>dotnet test tests/Nexus1.RootCause.ComponentTests</code> runs against real LocalDB databases (created and dropped per test). The H9 leading-question case appears as <i>Skipped</i> , with its reason, in every run.
The engine ignores seeded answer keys	<code>GraphWalkerDerivationTests.Walker_output_is_byte_identical_when_the_illustrative_fields_are_poisoned</code> — read it, run it, then try poisoning other fields.

4 Troubleshooting known setup friction

Every item here happened on the project's own machine — a Windows 10 laptop with an Intel i5-3340M, 16 GB of RAM, no GPU and no administrator rights — and is recorded in its runbooks or evidence reports.

4.1 SQL Server LocalDB

- **The instance name** is `(localdb)\mssqllocaldb` everywhere. It is SQL Server 2016 (13.0 SP1) on this machine — which is why the project computes cosine similarity in C# instead of using SQL Server 2025's vector features (ADR-032).
- **Migrations must run before the SQL logins are created.** The login scripts do `USE AlarmManagementDb`, `USE RootCauseDb` and so on, and those databases only exist once migrations have created them. Earlier revisions of the provisioning runbook listed the logins first; it now puts migrations first, and both login runbooks state the prerequisite (§6.3).
- **Use the Infrastructure project as the EF startup project.** `dotnet ef ... --startup-project src/Hosts/...` fails with *"doesn't reference Microsoft.EntityFrameworkCore.Design"*. Every `*.Infrastructure` project references the design package and carries its own design-time factory.
- **Pending migrations make readiness 503.** `DbContextHealthCheck<T>` checks connectivity *and* that the context's migrations are applied (ADR-018), using that context's own history table (`__EFMigrationsHistory_<Context>`). A "reachable but missing N migration(s)" message means migrations were not run — or, as ADR-044 found, that a context was configured with the wrong history-table name.
- **sqlcmd on this box:** the provisioning runbook notes that `-i` needs an absolute Windows path. The developer's Windows login is sysadmin on LocalDB; use `-E` (trusted) for administration and the scoped logins only to test their permissions.
- **Transient LocalDB command timeouts** appear under heavy concurrent load. See §4.4 — the fix is less concurrency, not bigger timeouts.

4.2 RabbitMQ

- **Use Erlang/OTP 27, not the latest.** RabbitMQ 4.3.4 failed to boot on OTP 29.0.5 with `BOOT FAILED {incompatible_feature_flags, ...}`. The portable OTP 27.3.4.16 works, pointed at through `ERLANG_HOME` (`docs/runbooks/local-rabbitmq.md`).
- **The management plugin is required, on port 15672.** Consumers apply their dead-letter policy through the management HTTP API at startup, and the port is hard-coded (`RabbitMQOptions.ManagementPort = 15672`, not read from configuration). Enable it with `rabbitmq-plugins enable rabbitmq_management`.
- **guest / guest works only over loopback** (RabbitMQ's default). That is fine locally and will fail from another machine or a container.
- **New shells need the environment variables.** `ERLANG_HOME` and `RABBITMQ_BASE` are set at user level; an already-open shell will not see them.
- **If the broker is down, the hosts still start** (ADR-044). Look for `RabbitMQ unreachable ... retrying in ...` in the log, and `Connected to RabbitMQ ... on attempt N` once it appears. A credentials problem is logged distinctly: *"RabbitMQ rejected the configured credentials ..."*. The outbox relay logs an error every 250 ms while the broker is down — known noise, not fixed.

4.3 Ollama and the 100–150 s cold start

- **A cold call takes about 93–150 s on CPU** (model load plus first generation); warm calls take roughly 20–70 s. On 2026-09-25 a cold 0418 call took 105 s and a warm 0419 call 71 s.

- **Timeouts are layered on purpose:** the host's chat call allows 150 s (`Ollama:RequestTimeoutSeconds`), the BFF's hop 180 s, and the live E2E test 180 s. The host fails first, so a slow model surfaces as the host's own 503, not a BFF 502.
- **Make cold starts rare:** set `OLLAMA_KEEP_ALIVE` (e.g. `30m`, or `-1` for indefinitely) on the Ollama process. It does not remove the first cold call after Ollama restarts.
- **Readiness** `ollama ... model(s) not installed` means the server answers but `nexus-dslm` or `nomic-embed-text` is missing: build or pull it (§6.5). A Healthy `ollama` check does *not* mean the model is loaded.
- **"Still missing an embedding" after provisioning** means Ollama or `nomic-embed-text` was unavailable. Lexical retrieval still works, and the semantic path wakes up after you rerun `-- provision`.
- **Ollama updates itself.** ADR-033 pinned Ollama 0.34.2; this machine now runs 0.34.3. The model is pinned by its base blob digest (`sha256:5ee4f07c...`), which is what reproducibility rests on.

4.4 Memory contention on modest machines

A 3B model needs about 2 GB of contiguous RAM, and each test assembly plus LocalDB adds more. The project hit this for real:

The H9 slice's parallel-run incident (`2026-09-24-h9-evaluation-harness-core.md`). `dotnet test Nexus1.Runtime.sln` ran all assemblies at once: 34 passed, 3 failed. EmergencyPreparedness and ReinforcementLearning component tests hit `SqlException: Execution Timeout Expired`, and two live Ollama tests got HTTP 500 — Ollama's own `server.log` read *"failed to allocate CPU buffer of size 1923946496 ... unable to allocate CPU buffer"*. Every assembly passed when rerun in isolation. The timed-out runs also left three test databases behind, which were dropped by hand.

- **Run test assemblies one at a time** when the live model tier is enabled. Every slice since H9 does exactly that for its regression gate.
- **Do not run the full E2E stack alongside a full test run.** Start the stack after the tests finish.
- **Leftover** `*ComponentTests_<guid>` **databases** after an interrupted run can be listed with `SELECT name FROM sys.databases WHERE name LIKE '%Tests[_]%'` and dropped once you have confirmed they are yours.
- An earlier observability campaign recorded the same class of problem: a transient LocalDB command timeout under heavy concurrent polling (`2026-08-16-observability-final-chain-proof.md`).

4.5 Observability stack

- **Prometheus 3.x keeps dotted OpenTelemetry names** (`nexus1.diagnosis.*`), which Grafana 10 cannot query. `docs/observability/prometheus.yml` therefore sets `metric_name_validation_scheme: legacy`, giving the underscore names the dashboard uses. If you changed the scheme after scraping, wipe the TSDB directory.
- **The** `or vector(0)` **trap:** on a *labelled* metric it returns the real series and a phantom empty-label `0`, so a stat panel shows two numbers. The committed dashboard wraps these in `sum(... or vector(0))` (ADR-038 evidence).
- **The dashboard provider path is absolute** (`grafana/provisioning/dashboards/nexus1.yml`); adjust it to your checkout.

4.6 Hosts, BFF and console

- **"Missing ConnectionStrings:... configuration"** at startup means that host's User Secrets are not set, or the host is not running in the Development environment (User Secrets load only there). See §6.4.
- **The BFF returns 502 on a RootCause route** when `RootCause.Host` is unreachable; a relayed **503** means the host ran but a dependency (Ollama, a database) failed. Both bodies carry a `traceId` to search for in the log.

- **The console's calls are blocked by CORS** unless the BFF runs in Development and the console is on exactly `http://localhost:4200`.
- **Playwright cannot find** `@playwright/test` when launched from some shells. Setting `NODE_PATH` to `console/nexus-console/node_modules` fixed it here.
- **The BFF's Overview endpoint** resolves several contexts at once and fails as a whole if one is not composed. When using `BffContexts__Enabled__N` subsets, include the contexts the screen needs (`console/nexus-console/e2e/README.md`).

5 The honest-gap discipline, as philosophy

"Nothing claims to exist that does not. A diagram, a 'done' checkbox, or a status message is valid only when it matches the repository, the contracts, the data ownership, the tests, and the observable runtime behavior." — the project's standing rule (`CLAUDE.md`).

It would be easy to read that as house style. It is closer to a load-bearing requirement, for a reason specific to what this repository is. NEXUS-1 is a public teaching reference backing a book series — cloned, in the project owner's words, by more than two hundred readers. Every one of them will lift patterns out of it: a readiness check, a retry loop, a way of wiring a local model. A reference implementation that overstates what it does teaches the overstatement along with the code. So every claim has to be checkable, and every gap has to be named where a reader will trip over it — not tucked into a backlog.

Three examples from this repository's own history show what that means in practice.

The narration is not deterministic, and the project says so

The book's H6 control asks for determinism. Temperature 0 and a fixed seed *look* deterministic, and it would have been easy to assert that. When ADR-033 measured it, the engine's verdict was identical run after run — but the model's narration was not byte-identical, because multi-threaded float reductions on a CPU vary in order. The project recorded exactly that: the *decision* is the deterministic guarantee, the prose is not. The consequence is visible in Section 3: two runs of the same incident seal different hashes over the same verdict, and nobody is surprised, because it was never claimed otherwise.

Reachable is not warm

ADR-044's Ollama readiness check calls `/api/tags`. A green result means the server answers and both models are installed — and *nothing more*, because `/api/tags` never loads a model. The easy thing would have been a check named "ollama-ready" that let everyone assume the model was hot. Instead the check's own Healthy description says it is not a warmth check, and the slice proved the distinction live: with the model unloaded (`ollama ps` empty before and after), readiness stayed 200. A reader copying the pattern copies the caveat with it.

A detection rule that ships with a map of its blind spots

ADR-043 added a rule that makes the engine abstain when the evidence says its causal graph is incomplete. The rule is genuinely useful, and the ADR spends as much space on what it *cannot* do: a silent true cause, a spurious edge whose timing happens to fit, a missing downstream edge, and the rule's own cost — a standing alarm could trigger a false abstention. The same slice also refused an easier win: its test of the old engine "passed" by abstaining with *no grounding*, only because the fixture had no corpus. The evidence names that as accidental rather than counting it as detection.

Why gaps are named in the artifact itself

Naming a gap costs a sentence. Hiding one costs every reader who builds on the unstated assumption. That is why the console labels its advisory panel "not running in this build"; why EVT-2026-0420's `404` is treated as evidence rather than a bug; why the metric is called `grounding_hit` and not `recall`; and why this guide opens with the three places the brief it was written from disagreed with the repository. The discipline applies to the process too: the ADR-044 slice records two mistakes its implementer made and corrected — a timeout regression and a password printed into tool output — because leaving them out would have been the same kind of overclaim the project exists to avoid.

6 Installation and running guide

This consolidates the repository's runbooks (`docs/runbooks/`) into one ordered procedure, updated for the current secrets setup (User Secrets, ADR-044) and corrected where the runbooks are known to be wrong.

Tested scope. This procedure is proven on one machine: Windows 10, .NET SDK 8.0.425, SQL Server 2016 LocalDB, Node 22.19 / npm 10.9.3, Ollama 0.34.3, CPU-only with 16 GB of RAM, and *no administrator rights* — which is why RabbitMQ, Prometheus, Grafana and the collector run as portable binaries rather than services. Linux and macOS are untested (LocalDB is Windows-only), and there are no container images for the .NET hosts.

6.1 Prerequisites

Component	Version used here	Notes
.NET SDK	8.0.424+ (<code>global.json</code> , <code>latestPatch</code>)	Also restores the local <code>dotnet-ef</code> 8.0.11 tool
SQL Server LocalDB	2016 SP1 (13.0.4001)	Instance <code>(localdb)\mssqllocaldb</code> ; <code>sqlcmd</code> on PATH
Erlang/OTP	27.3.4.16 (portable zip)	Not OTP 29 (\$4.2)
RabbitMQ	4.3.4 (portable)	With the management plugin
Ollama	0.34.x, user-scope install	Loopback server <code>127.0.0.1:11434</code>
Node.js / npm	22.19 / 10.9.3	For the Angular 18 console
Prometheus / Grafana (<i>optional</i>)	3.14.0 / 10.2.3 OSS (portable)	Diagnosis metrics only
OTel Collector (<i>optional</i>)	otelcol-contrib 0.158.0	Tracing and messaging-metrics campaigns
Hardware	16 GB RAM strongly advised	A 3B model on CPU plus LocalDB and test hosts (\$4.4)

6.2 Clone, restore, build

```
git clone <repository-url> nexus1 && cd nexus1
dotnet tool restore                # dotnet-ef 8.0.11 from .config/dotnet-tools.json
dotnet build Nexus1.Runtime.sln -c Debug # expect: 0 Warning(s) 0 Error(s)
```

6.3 Databases: migrations first, then the scoped logins

Step 1 — apply every context's migrations with your own Windows login. Each `*.Infrastructure` project's design-time factory uses `Trusted_Connection=True` and creates its database if needed. The Infrastructure project must be both `--project` and `--startup-project` (verified on this machine: the host-as-startup form fails with "doesn't reference Microsoft.EntityFrameworkCore.Design"; this form prints "No migrations were applied. The database is already up to date." on a migrated database).

```
# PowerShell - all 17 contexts (7 physical databases)
$contexts = "AlarmManagement","ReactorFleet","CorePlatform","Instrumentation","DigitalTwin","Maintenance",
            "EventManager","Robotics","RadiationMonitoring","EmergencyPreparedness","ReinforcementLearning",
            "Security","Organization","Audit","Compliance","Reporting","RootCause"
foreach ($c in $contexts) {
    $p = "src/Contexts/$c/Nexus1.$c.Infrastructure"
    dotnet ef database update --project $p --startup-project $p
}
```

Each context records its own history in `dbo.__EFMigrationsHistory_<Context>`. That is how eleven contexts share `AlarmManagementDb` without colliding (ADR-006, 020–026).

Step 2 — create the two scoped SQL logins, as your own sysadmin Windows login, using the T-SQL in the two runbooks (save each block as a `.sql` file and pass an absolute path to `-i`):

- `docs/runbooks/local-scoped-sql-login.md` — `nexus1_app`: DML on exactly the schemas each database's contexts write, `SELECT` on `dbo` for the history tables, no DDL (ADR-028).
- `docs/runbooks/local-explain-readonly-sql-login.md` — `nexus1_explain`: `db_datareader` on RootCauseDb plus explicit `DENY` on every write/execute verb (H7).

Choose your own local passwords where the runbooks show `<your local ... password>`. They never go into a tracked file.

6.4 Connection strings in User Secrets

The hosts read their connection strings from each project's User Secrets store, which ASP.NET Core loads in the **Development** environment. The tracked `appsettings.json` files contain none, and an architecture test fails if a password is ever committed there (ADR-044).

```
# RootCause.Host — both logins
dotnet user-secrets set "ConnectionStrings:RootCauseDb" "Server=(localdb)\mssqllocaldb;Database=RootCauseDb;User
Id=nexus1_app;Password=<your nexus1_app password>;" --project src/Hosts/Nexus1.RootCause.Host
dotnet user-secrets set "ConnectionStrings:RootCauseExplainDb" "Server=
(localdb)\mssqllocaldb;Database=RootCauseDb;User Id=nexus1_explain;Password=<your nexus1_explain password>;" --
project src/Hosts/Nexus1.RootCause.Host
```

```
# ModularRuntime and BFF — the same six keys each, all as nexus1_app
foreach ($h in "Nexus1.ModularRuntime","Nexus1.Bff") {
  foreach ($db in "AlarmManagementDb","AuditDb","ComplianceDb","ReportingDb","SecurityDb","OrganizationDb") {
    dotnet user-secrets set "ConnectionStrings:$db" "Server=(localdb)\mssqllocaldb;Database=$db;User
Id=nexus1_app;Password=<your nexus1_app password>;" --project "src/Hosts/$h"
  }
}
dotnet user-secrets list --project src/Hosts/Nexus1.Bff      # check which keys a host has
```

The BFF also accepts an optional `ConnectionStrings:RadiationMonitoringDb` override; without it, RadiationMonitoring uses `AlarmManagementDb`. RabbitMQ's `guest`/`guest` development credentials remain in `appsettings.json` — out of ADR-044's scope.

History note. Earlier revisions committed the local development passwords in plaintext. ADR-044 moved them out of the tracked files without rotating them, so the old values remain in git history. Use your own values; never reuse the historical ones anywhere.

6.5 Ollama: models and the pinned `nexus-dslm`

```
ollama pull qwen2.5:3b-instruct-q4_K_M      # base of nexus-dslm (standard Ollama command)
ollama pull nomic-embed-text                # embedding model
ollama create nexus-dslm -f src/Contexts/RootCause/Nexus1.RootCause.Explain/nexus-dslm.Modelfile
ollama show nexus-dslm --modelfile          # record the resolved base digest
ollama list                                 # expect nexus-dslm, nomic-embed-text
```

- **Pin by digest, not by tag.** ADR-033's reproducibility record: base blob `sha256:5ee4f07cdb9beadbbb293e85803c569b01bd37ed059d2715faa7bb405f31caa6`; `nomic-embed-text` id `0a109f422b47`.
- **Keep the server on loopback** (`127.0.0.1:11434`, Ollama's default). Models are pulled once while online; inference needs no network. Optionally set `OLLAMA_KEEP_ALIVE=30m` before starting Ollama.

- **GAP** The strongest air-gap step — disabling the network adapter during inference — needs administrator rights and was not performed. Loopback binding plus captured zero egress is the evidence (ADR-033).

6.6 Provision RootCauseDb (seed + embeddings)

The diagnosis route never seeds on startup. It assumes the data exists. Provisioning is an explicit, idempotent command: it seeds the 0418 and 0419 grounding fixtures, fills the corpus embeddings through `nomic-embed-text`, and creates the fixed EVT-2026-0420 Inconclusive case, then exits.

```
dotnet run --project src/Hosts/Nexus1.RootCause.Host -- provision # Development via launchSettings
```

On a fresh database, expect `Seed complete: 17 components, 4 corpus chunks.`, `Embedding ingest complete: 4 newly embedded, 0 still missing an embedding.` and `Provisioned EVT-2026-0420 QA-escape case as Inconclusive (AnalysisId 20260420, unit 1).` A re-run is a no-op: `0 newly embedded` and `... already provisioned` (verified 2026-09-25). Earlier revisions of the runbook showed `10 components, 2 corpus chunks` — the counts from before EVT-2026-0419 — and were corrected alongside this guide. The 0420 case's two integration events wait in RootCause's outbox and are published when RootCause.Host runs with the broker up.

6.7 RabbitMQ (portable)

```
$env:ERLANG_HOME = "$env:LOCALAPPDATA\erlang-otp27"
$env:RABBITMQ_BASE = "$env:LOCALAPPDATA\rabbitmq\data"
& "$env:LOCALAPPDATA\rabbitmq\rabbitmq_server-4.3.4\sbin\rabbitmq-plugins.bat" enable rabbitmq_management #
once
& "$env:LOCALAPPDATA\rabbitmq\rabbitmq_server-4.3.4\sbin\rabbitmq-server.bat" # run
in background
& "$env:LOCALAPPDATA\rabbitmq\rabbitmq_server-4.3.4\sbin\rabbitmqctl.bat" status #
exit 0 = up
```

Management UI and API: `http://localhost:15672` (`guest` / `guest`, loopback only). Stop the broker with `rabbitmqctl stop`. There is no Windows service.

6.8 Start the hosts and the console

Order matters less than it used to: since ADR-044 the broker can come up before or after the hosts. The only hard dependency is that the BFF's RootCause routes need RootCause.Host.

```
# Each in its own shell. Each host's Properties/launchSettings.json supplies its port
# (5102 / 5101 / 5103) and DOTNET_ENVIRONMENT=Development, so User Secrets load.
dotnet run --project src/Hosts/Nexus1.RootCause.Host
dotnet run --project src/Hosts/Nexus1.ModularRuntime
dotnet run --project src/Hosts/Nexus1.Bff
# (Services:RootCauseHost defaults to http://localhost:5102 in the BFF's appsettings.json)

cd console/nexus-console
npm install # first time
npm start # → http://localhost:4200
```

```
# Health
curl http://localhost:5102/health/ready # rootcause-db, rootcause-explain-db, ollama
curl http://localhost:5101/health/ready # 16 context databases (+ migrations applied)
curl http://localhost:5103/health/ready # the BFF's composed contexts
```

Then open `http://localhost:4200/rcgraph`. The graph appears at once (a plain database read), and the verdict fills in when the diagnosis finishes (20–150 s).

To compose only some contexts in the BFF (less memory, faster start), set `BffContexts__Enabled__0=ReactorFleet`, `__1=...` and so on. The E2E README uses `ReactorFleet`, `Instrumentation`, `AlarmManagement` and `RadiationMonitoring`.

6.9 Optional: Prometheus + Grafana (Appendix I)

```
& "$env:LOCALAPPDATA\prometheus\prometheus-3.14.0.windows-amd64\prometheus.exe" `
  --config.file="<repo>\docs\observability\prometheus.yml" `
  --storage.tsdb.path="$env:LOCALAPPDATA\prometheus\data" --web.listen-address="localhost:9090"

$env:GF_PATHS_PROVISIONING = "<repo>\docs\observability\grafana\provisioning"
$env:GF_AUTH_ANONYMOUS_ENABLED = "true"; $env:GF_AUTH_ANONYMOUS_ORG_ROLE = "Admin" # dev only
& "$env:LOCALAPPDATA\grafana\grafana-v10.2.3\bin\grafana-server.exe" --homepath
"$env:LOCALAPPDATA\grafana\grafana-v10.2.3"
```

Prometheus scrapes `localhost:5102/metrics` (job `nexus1-rootcause`). Grafana at `:3000` provisions the Prometheus datasource and the "NEXUS-1 Diagnosis (Appendix I)" dashboard. Edit the absolute `path:` in `grafana/provisioning/dashboards/nexus1.yml` first. Run a few diagnoses to see data: `nexus1_diagnosis_abstentions`, `_validator_rejections`, `_duration`, `_grounding_hit` (a proxy, not recall). The OTEL collector for tracing campaigns is documented separately in `docs/runbooks/local-otel-collector.md`.

6.10 Run the tests

```
# One assembly at a time is the reliable gate on a modest machine ($4.4)
Get-ChildItem tests -Directory | ForEach-Object { dotnet test $_.FullName --no-build }

dotnet test tests/Nexus1.ArchitectureTests           # dependency law, served-model rule, secrets guard
dotnet test tests/Nexus1.RootCause.ComponentTests    # real LocalDB; H9 live tier skips if Ollama is absent

cd console/nexus-console
npm test                                             # Jest
npx playwright test --grep-invert "@slow"          # fast E2E: BFF + console + LocalDB
npx playwright test --grep "@slow"                 # live RAG E2E: full stack incl. Ollama
```

At this commit the regression gate is: build with 0 warnings and 0 errors; 36 test assemblies green when run in isolation; `ArchitectureTests` 9/9; `RootCause.ComponentTests` 93 tests (92 passed, 1 skipped by name); the `@slow` E2E 2/2 (ADR-044 evidence). `operator-session.e2e.ts` acknowledges a real unit-1 alarm on each run, so it is not idempotent across unlimited reruns.

6.11 Shut down

Stop each `dotnet run` and `npm start` with `Ctrl+C`, the broker with `rabbitmqctl stop`, and Prometheus and Grafana by the process listening on their port (`Get-NetTCPConnection -LocalPort 9090 -State Listen` → `Stop-Process`). Nothing here installs a Windows service.

6.12 What is not built yet — consolidated

- Container images for the .NET hosts; Kubernetes manifests; any non-LocalDB database target.
- Authentication and authorization on the HTTP surfaces; a production secret store (only User Secrets for development).
- True `retrieval_recall` (Appendix J) and a labelled evaluation set of real size; the CauseTag-vs-origin check behind the skipped H9 leading-question trap; a free-form question interface.
- Audit and Compliance consumers for `RootCauseCaseInconclusiveV1` (the ADR-040 follow-on).
- BFF observability and edge metrics (`nexus1.edge.requests`), `nexus1.projection.lag` , Prometheus recording rules and alerting.
- Contract tests and a distributed-slice E2E test project with real tests (both projects exist but are empty).
- UI wording for the post-ADR-042 weights ("Illustrative weight/confidence" labels).

Appendices

A · ADR index

ADR	Decision
001-amend	ReactorFleet included in Phase 1, in-process only
002-amend	ADR-002 topology stands; CQRS markers in BuildingBlocks.Application
003 – 005	Phase 1 domain slices: ReactorFleet, AlarmManagement (flood policy), RootCause
006	ReactorFleet shares AlarmManagement's database
007	Hosts get health-only Minimal APIs; Query BFF deferred
008 – 009	Messaging backbone: topology, outbox/inbox; retry, backoff, dead-letter
010 – 012	Fan-out subscribers: Audit, Compliance, Reporting
013 – 014	Observability: tracing foundation; metrics foundation
015 – 026	Phase 2 sectors 1–11 (CorePlatform ... ReinforcementLearning); 018: health checks verify migrations
027	OpenTelemetry deferred for the eleven Phase 2 sectors
028	Scoped SQL login (nexus1_app)
029	Licensed source PDFs purged from the repo and its history
030 – 031	BFF walking skeleton; "latest per parent" LINQ pattern
032 – 044	The RootCause/RAG phase (§2.8)

B · RAG evidence reports (artifacts/evidence/)

Report	Slice
2026-09-07-rootcause-grounding-walking-skeleton.md	ADR-032 deterministic engine
2026-09-22-rootcause-explain-airgapped-e2e.md	ADR-033 Explain stage, air gap
2026-09-22-rootcause-diagnosis-query-route.md	ADR-034 HTTP route, audit-hash match, H7 live
2026-09-22-bff-rootcause-http-hop.md	ADR-035 BFF hop
2026-09-22-rootcause-host-cold-timeout-hardening.md	ADR-033 addendum, 150 s timeout
2026-09-23-ch29-root-cause-angular-screens.md	Ch. 29 screens
2026-09-23-ch24-ai-diagnostics-disclaimer-reconcile.md	Disclaimer correction
2026-09-23-rootcause-incident-graph-endpoint.md	ADR-036 graph endpoint
2026-09-24-rootcause-cross-screen-live-e2e.md	Ch. 33 live E2E
2026-09-24-second-fixed-incident-0419.md	ADR-037 EVT-2026-0419
2026-09-24-observability-prometheus-grafana.md	ADR-038 metrics
2026-09-24-rootcause-inconclusive-state.md	ADR-039 Inconclusive
2026-09-24-evt-0420-qa-escape-content.md	ADR-040 EVT-2026-0420
2026-09-24-h9-evaluation-harness-core.md	ADR-041 H9 harness (and the parallel-run incident)
2026-09-24-adr-042-candidates-from-observables.md	ADR-042 de-circularization

Report	Slice
<code>2026-09-25-adr-043-unexplained-precursor.md</code>	ADR-043 model mismatch
<code>2026-09-25-adr-044-startup-readiness-secrets.md</code>	ADR-044 hardening

C · Glossary

Abstention	A run that ends without a verdict and with a named reason. A first-class outcome (HTTP 200), sealed like a verdict.
Attribution share	A candidate's weight: the share of the flood it newly explained in the greedy ranking (ADR-042). Not a probability.
Artefact edge	An edge from a source that corrupts measurements (e.g. EMI from breaker BKR-2A) into the channels it couples into.
Backbone edge	An engineered cascade edge with a delay window; walked and corroborated.
Coverage	The share of the flood's alarms a candidate's reach explains (FV-104: 11/14).
Fixed incident	A seeded, fully known teaching incident in <code>IncidentRegistry</code> (0418, 0419).
Onset	A channel's earliest good-quality historian sample — the physical start of its disturbance, as opposed to alarm arrival time.
One-Truth Pipeline	The book's name for "the engine decides, the model explains", bounded by H1–H10.
Unexplained precursor	An alarmed component outside the origin's reach whose onset precedes the origin's (ADR-043).
Walking skeleton	A thin but complete end-to-end implementation, proven before any breadth is added.

This guide describes commit `1fbada7`. When the code moves, the ADRs, runbooks and evidence reports are the source of truth; regenerate this PDF from `docs/guide/nexus1-programmers-guide.html` with `docs/guide/build-guide-pdf.mjs`.